



INTERNSHIP LOGBOOK

Intern	Firas Mahjoubi
Company	INSOMEA
Address	Immeuble Lac d'Or "Business Center", Bloc B, 3 rd floor, Les Berges du Lac 2, Tunis 1053
Client / Project	End-to-End MLOps Platform (Training, Experiment Tracking & Model Deployment on Kubernetes)
Module	Platform foundations & development-environment setup
Period	12 January 2026 — 11 July 2026
Supervisors	Mr. Amine Gonji (company), Mr. Ben Mardes Achref (academic), Mr. Benarous Abderrahmen (technical)
School	Private Higher School of Engineering and Technology — ESPRIT
Programme	Software Engineering (Engineering Degree)

Project Context

The project consists of designing and implementing, within **INSOMEA**, an **end-to-end MLOps platform** that industrializes the life cycle of *Machine Learning* models. Today, model training, experiment tracking and deployment rely on manual, poorly reproducible practices: scripts run locally, results scattered across machines, and hand-crafted production rollouts.

The platform aims to unify this cycle: a user (*data scientist*) uploads their code and data, triggers an isolated training job on a **Kubernetes cluster**, and sees its metrics, parameters and artifacts captured automatically in a tracking server (**MLflow**). The resulting models can then be registered, versioned, deployed as inference services and queried, all from a single web interface.

My contribution during this first fortnight covers the **framing** of the project, the **state of the art** of MLOps tooling, and the **setup of the technical environment** that underpins all the sprints to come.

INTERNSHIP LOGBOOK N°1 (12/01/2026 — 23/01/2026)
Framing, MLOps state of the art, and environment setup

1. Tasks Accomplished

During this first fortnight, I laid the foundations of the project: framing the requirements, benchmarking the MLOps building blocks, and deploying a complete development environment that mirrors the target architecture.

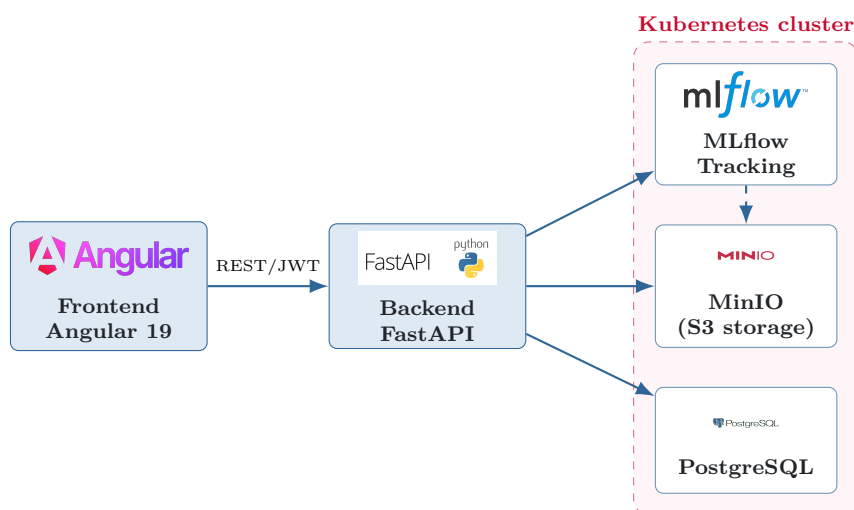
Framing and scope — I defined the functional scope of the platform (code upload, cluster-based training, automatic experiment tracking, model registry, deployment and prediction) and organized the work into four sprints following the **Scrum** methodology. The first sprint, *Foundation*, covers authentication and project management.

MLOps state of the art — I carried out a comparative study of the *cloud-native* ecosystem components in order to settle on a coherent, open-source stack: **MLflow** for experiment tracking and the model registry, **KServe** for inference serving on Kubernetes, **Kubeflow Pipelines / Argo** for training orchestration, **MinIO** as S3-compatible object storage, and **FastAPI / Angular** for the application.

Environment setup — I provisioned a **Kubernetes cluster** and deployed the **MLflow** tracking server on it (backed by a **PostgreSQL** database for metadata and by **MinIO** for artifacts). I initialized the **FastAPI** backend skeleton (asynchronous, SQLAlchemy) and the **Angular 19** frontend scaffold.

Initial design and start of Sprint 1 — I modelled the two founding entities (User, Project), settled on a microservices architecture, and chose **stateless JWT** authentication. I started implementing authentication (sign-up, sign-in, session refresh) with **bcrypt** password hashing.

Technical environment set up:



Components set up:

- **Kubernetes cluster:** namespaces, `kubectl` access, basic ingress.
- **MLflow:** tracking server with a PostgreSQL backend and a MinIO *artifact store*.
- **FastAPI backend:** project structure, configuration, asynchronous database connection, authentication endpoints (JWT).
- **Angular 19 frontend:** *scaffolding*, routing, sign-in/sign-up screens, `AuthService` and a *route guard*.
- **Git repository** and a basic continuous-integration pipeline (GitHub Actions).

2. Deliverables Produced

- **Framing document** and **MLOps state-of-the-art study** (technical-stack decision).
- **Working development environment**: Kubernetes cluster + MLflow + MinIO + PostgreSQL.
- **FastAPI backend skeleton** with JWT authentication (sign-up, sign-in, refresh).
- **Angular frontend skeleton** with the authentication screens.
- **Initial data model** (User, Project) and architecture diagram.

3. Skills Applied and Lessons Learned

This phase allowed me to consolidate an overall vision of **MLOps engineering** and of setting up a *cloud-native* environment.

- **Containerization & orchestration**: Docker, Kubernetes (deployments, services, namespaces).
- **MLOps ecosystem**: MLflow (tracking & registry), MinIO (S3 object storage), PostgreSQL.
- **Backend**: asynchronous FastAPI, SQLAlchemy, OAuth2/JWT security, bcrypt hashing.
- **Frontend**: Angular 19 (*standalone* components), Tailwind CSS, session management.
- **Methodology**: Scrum organization, breakdown into sprints and backlog.

4. Difficulties and Solutions

- **Access to the cluster's internal services** (MLflow, MinIO) from outside: solved by setting up *ingress* and *port-forward*, together with dedicated environment variables.
- **MLflow ↔ MinIO integration** (artifacts on S3 storage): solved by configuring the S3 *credentials* and the MinIO *endpoint* on the tracking server side.
- **Stateless authentication**: chose an *access/refresh* token pair and set up an Angular interceptor for transparent session refresh.
- **Environment reproducibility**: versioned Kubernetes manifests and a *docker-compose* setup for local execution.

5. Reflection and Alignment with My Expectations

This first fortnight, largely dedicated to framing and environment setup, was particularly instructive: it forced me to take a step back over the whole MLOps chain before writing a single line of business code. Having a solid, reproducible environment from the outset is, I believe, the condition for the success of the following sprints.

The work carried out is fully aligned with my expectations: it let me build up skills in the *cloud-native* ecosystem and validate the architectural choices with my supervisors. I therefore approach the first functional sprint (authentication and project management) on clear, well-mastered foundations.